APS-DDA / TECHNICAL PUBLICATION

APS-DDA v0.1

Proposed Agent Participation Standard for Data, Decisions and Actions

Kunal Yadav

3 November 2026 · Proposed v0.1 · Working draft

Normative working draft retained from 16 October 2026. This publication adds a cover and visible references, not a normative amendment or ratification. The later .NET/C# implementation supports selected requirements; its tested scope is documented separately.

Copyright © 2026 Kunal Yadav. Authored document prose and diagrams: CC BY 4.0. Third-party material retains its original terms.

Contents

1. Status, conventions, and purpose 2
2. Scope and minimum profile 2
3. Roles and conformance claims 3
4. Artifact model and structural schema 4
5. Contracts, meaning, discovery, and trust 4
6. Execution Context and permission intersection 6
7. Policy Decisions, Evidence, and Decision Specifications 6
8. Execution Plan and canonicalization 7
9. Authority, approval, expiry, and revocation 8
10. Source state, idempotency, and outcome recovery 8
11. Lifecycle, cancellation, compensation, and errors 9
12. Privacy, residency, logging, and untrusted content 10
13. Application Specifications and builder prompts 10
14. Bindings and compatibility 10
15. Conformance tests and non-guarantees 11
16. Implementation boundary 12
References 12

WORKING DRAFT — 16 October 2026

Status: Prepared for review. This is a proposed profile, not a ratified standard, implementation, certification, security assessment, or empirical validation. It makes no claim of name uniqueness, priority, adoption, or universal correctness.

1. Status, conventions, and purpose

APS-DDA means **Agent Participation Standard for Data, Decisions and Actions**. Version 0.1 is an implementation-oriented proposed profile for the externally observable behavior needed to compose source-owned data products and source-owned business capabilities into a governed operational loop. It specifies a candidate contract model and normative obligations, not an implementation-complete wire binding or a validated conformance program. Its intended outcome is to avoid a **new bespoke analytical replica and disconnected decision/action application** for a supported operational workflow where source products and capabilities can meet the stated requirements.

APS-DDA is a behavioral and semantic standard carried over HTTPS. It is **not** a networking layer, not a new general messaging protocol, not an identity system, not a universal ontology, and not a requirement to use an LLM. A participant may be an LLM agent, a deterministic adapter, a rules service, a workflow engine, a vendor-native platform component, or a conventional service. What matters is its published contract and observable conformance behavior.

The key words **MUST**, **MUST NOT**, **REQUIRED**, **SHOULD**, **SHOULD NOT**, and **MAY** in this document are to be interpreted as described in BCP 14 when, and only when, they appear in all capitals. 1 2

This profile standardizes records, mandatory checks, and failure behavior. It does not assert that a record proves a business fact, that a signature makes source content safe instructions, that a decision is wise, or that a distributed workflow is atomic. A structural schema is necessary for interoperable shape but not sufficient for runtime correctness.

2. Scope and minimum profile

The v0.1 minimum viable profile addresses a single tenant using trusted publishers, bounded data-product reads, named source commands, a finite command DAG, immutable evidence/plan/receipt links, either short-lived human approval or tightly bounded policy authority, and operation polling/reconciliation. A pilot MUST select one domain profile and state its owner, source systems, entity identifiers, risk tiers, approval roles, policy mechanism, residency conditions, and recovery owners.

The v0.1 core deliberately excludes arbitrary federated SQL, unconstrained source scans, a universal ontology, free-form semantic bargaining, cross-source atomic commit, an external global registry, automated certification, universal analytics replacement, and arbitrary generated-code execution. A participant MUST refuse an unsupported profile or scope using a typed error or counteroffer. It MUST NOT silently reinterpret a required business constraint or downgrade a critical control.

The profile does not prohibit governed copies, caches, read models, history stores, identity products, or replication. A declared materialized product is a valid participant. It MUST disclose its source contract/semantic versions, transformation version, watermark or as-of time, policy/classification, region, retention, invalidation/revocation behavior, and owner. The profile is not a zero-copy guarantee.

3. Roles and conformance claims

A Capability Manifest declares one or more of the following combinable roles. An implementation MUST NOT claim a role unless it provides the required artifacts and behavior.

3.1 APS-Read-Publisher

An **APS-Read-Publisher** MUST publish a versioned Data Product Contract and a manifest that states profile versions, bindings, endpoint/audience requirements, capability references, limits, residency, expiration, and deprecation condition. It MUST enforce source-side purpose, entitlement, field scope, query budget, rate/availability controls, and contract-defined retention. It MUST accept only the published bounded query shapes, or return `NEGOTIATION_REQUIRED`, `QUOTA_EXCEEDED`, or another safe error.

For every delivered evidence item, the publisher MUST identify the product contract version/digest, semantic bindings, query or compiled-plan digest, observation time, source version/ETag or equivalent where supplied, watermark/snapshot indicator where available, quality state, and cache/materialization status. It MUST NOT label heterogeneous reads as a coordinated snapshot unless its documented and tested contract covers the named sources.

3.2 APS-Action-Executor

An **APS-Action-Executor** MUST publish a Process Capability Contract for every consequential command. It MUST independently validate the request's source audience, subject/delegation, tenant, purpose, entitlement, policy/approval state, schema, capability version, exact plan binding, state precondition, idempotency rule, domain invariants, risk gates, and time bounds immediately before effect. It MUST fail closed if a required check cannot be completed.

The executor MUST be the source system or a source-authorized adapter that the source treats as part of its enforcement boundary. It MUST issue immutable source-issued Operation Receipts and expose the published reconciliation path. A runner, orchestrator, or UI MUST NOT claim that it committed source state solely because it sent a request or received a transport success response.

3.3 APS-Orchestrator

An **APS-Orchestrator** MUST resolve only tenant-trusted, pinned contracts. It MUST create or preserve the links among Task, Execution Context, evidence, Decision Spec, Execution Plan, authority, invocation, receipt, and terminal state. It MAY collect intent, seek clarification, select an allowed product, propose an alternative, render an explanation, or coordinate an invocation. It MUST NOT bypass the source executor, broaden a query or action scope, treat untrusted text as instructions, or report an unverified outcome as completed.

The orchestrator MUST durably record the plan digest, authority references, invocation attempts, source receipt references, reconciliation evidence, and terminal decision before it announces a terminal workflow result. Restart or replay MUST NOT change a plan core, mint a new side effect, or convert an unknown effect into a completed effect.

3.4 APS-Renderer

An **APS-Renderer** MUST render an Application Spec using the declared component catalog and typed bindings. It MUST visibly distinguish evidence, source timestamps/quality, uncertainty, proposal, approval, and verified result. It MUST emit a typed input or approval event to the authority/execution service. It MUST NOT turn arbitrary markup, generated code, builder prompt text, or a UI boolean into execution authority. It MUST NOT receive broad source credentials solely to render an approval control.

3.5 Conformance labels

`APS-Read` means conformance to applicable publisher/orchestrator/renderer obligations and grants no mutation privilege. `APS-Action` additionally requires an action executor, source-side authority validation, plan-bound approval or permitted policy authority, idempotency/reconciliation behavior, and action-lifecycle tests. These are implementation conformance claims only. They are not certification, endorsement, or a statement that the participant's data or decisions are universally correct.

4. Artifact model and structural schema

The companion `aps-dda.schema.json` is one self-contained JSON Schema Draft 2020-12 bundle. It is a compound schema document with a discriminated root `oneOf` and `$defs` for `CapabilityManifest`, `DataProduct`, `ProcessCapability`, `Task`, `Negotiation`, `EvidenceBundle`, `DecisionSpec`, `ExecutionPlan`, `PolicyDecision`, `Authority`, `Approval`, `CommandInvocation`, `OperationReceipt`, `ApplicationSpec`, and `BuilderPrompt`. Each top-level embedded definition has a stable `$id` and declares the 2020-12 dialect. JSON Schema 2020-12 describes a document containing embedded schemas as a compound schema document and explains that embedded resources must be evaluated under their applicable dialect. [3]

Each issued artifact MUST contain an `artifactType`, a closed `header`, and a closed `body` except for a dedicated URI-keyed `extensions` object. The header contains UUID `id`, semantic-version `artifactVersion`, `specVersion` of `APS-DDA/0.1`, issuer URI, issue timestamp, and a declared SHA-256 content digest. Corrections MUST be new artifacts with new version/digest. They MUST NOT mutate historical issued content. Suspension, revocation, or deprecation MUST be a separate status assertion or registry result.

For an APS-DDA artifact, `artifact_core` MUST be the complete parsed artifact with only `/header/contentDigest` removed. The content digest MUST be `"sha256:" + lowercase_hex(SHA-256(UTF-8(JCS(artifact_core))))`. All other header fields, the artifact discriminator, the complete body and any body extensions are covered. Signatures, transport metadata and mutable status assertions MUST be detached from this immutable artifact. This projection prevents a digest from containing itself. External policy records or schema resources referenced by digest MUST declare their own byte/canonicalization rules; the artifact rule MUST NOT be silently applied to non-APS-DDA resources.

A reference used in a Decision Spec, Execution Plan, Authority, Approval, or Command Invocation MUST pin artifact type, ID, artifact version, and content digest. It MUST NOT use an unpinned `latest` reference. Credentials, bearer tokens, private keys, and raw secrets MUST NOT be put in APS-DDA artifacts, examples, URIs, prompts, or logs.

Schema validation proves only shape and local syntax. It cannot prove that a declared digest matches bytes, a signature is valid, a publisher is trusted, a semantic term is equivalent, evidence is fresh, an approval is unexpired or consumed, a lifecycle transition is allowed, a precondition matches source state, a policy is correct, or an external effect occurred. These are runtime MUSTs and behavioral/semantic test obligations.

5. Contracts, meaning, discovery, and trust

5.1 Capability Manifest

A Capability Manifest MUST identify issuer, role claims, supported APS-DDA profile versions, schema/binding versions, endpoint/audience allowlist, authentication requirements, trust-key references if used, capability references, semantic-domain references, limits, logging/classification commitments, extension

support, residency, expiry, and deprecation. Discovery MAY begin with a self-published HTTPS location, an MCP resource, or an A2A Agent Card. Discovery MUST NOT itself establish trust or authority.

For production action, a tenant-controlled resolver or registry MUST pin participant identity, approved HTTPS domains/audiences, allowed role/profile/binding, artifact digest/status, semantic-domain permission, and revocation/status source. Unknown or revoked publishers, profiles, schema versions, or critical extensions MUST produce a typed refusal. A source that supports a newer version MAY offer a compatible alternate version only if the requester explicitly accepts it before plan or authority creation.

5.2 Data Product Contract

A Data Product Contract MUST identify its owner and support route; authoritative, derived, or observational status; schema reference; semantic references; grain; entity keys and permissible identity mappings; valid-time and transaction-time interpretation; current/history/correction/delete/tombstone behavior; lineage; quality/freshness condition; classification; permitted purposes; entitlement condition; retention; residency; bounded query shapes; row/scan/time/cost/concurrency budgets; paging behavior; evidence profile; compatibility; and deprecation behavior.

A semantic reference MUST identify namespace URI, term URI, version, kind, and local binding. APS-DDA does not silently equate domain terms. A domain profile defines terms such as `PurchaseOrder` or `CancelPurchaseOrder`; a publisher binds them to local implementation and source rules. An ambiguous, incompatible, untrusted, or revoked semantic binding MUST block an action and SHOULD prevent a result from being described as grounded.

A publisher MUST declare whether a requested identity mapping is authoritative, permitted for the stated purpose, and subject to match confidence, consent, merge/split, survivorship, or deletion propagation limits. An orchestrator MUST NOT infer identity equivalence from similar values.

5.3 Process Capability Contract

A Process Capability Contract MUST identify its named business command, owner, command semantic binding, endpoint/audience, input/result schema, allowed transitions, relevant invariant references, resource scope, state precondition mechanism, idempotency domain/retention/duplicate behavior/operation lookup, authority requirements, risk tier, cancellation behavior, safe errors/timeouts, receipt requirements, and compensation or explicit non-compensability. A generic mutable resource endpoint is not a sufficient process capability contract.

Each executor MUST enforce its own access, separation-of-duties, and business invariants. An external policy decision, an LLM result, or a valid schema MUST NOT bypass source rules. Named commands make business effects inspectable; they do not guarantee that different sources implement an identically named command identically.

5.4 Negotiation

A Negotiation artifact MAY counteroffer product/capability selection, projection/predicate/aggregation within the product contract, freshness, timing, declared cost/quantity scope, rate/capacity, region, or a declared alternate command path. It MUST list changed fields, reasons, expiration, and acceptance condition. It MUST NOT relax a source invariant, entitlement, classification, purpose restriction, segregation-of-duties requirement, hard risk threshold, required evidence, or current-state precondition. Accepted changes create a new immutable plan; a material post-approval change requires new authority.

6. Execution Context and permission intersection

Every read, plan, authority, invocation, and receipt MUST carry or reference an Execution Context. It MUST contain tenant ID, authenticated subject ID, authenticated workload/client ID, executor identity, optional delegation reference, purpose, classification/residency context where relevant, correlation UUID, and immutable policy-rule references. In v0.1, a plan's `context.correlationId` is its durable workflow identity and MUST be preserved by its authority and invocations.

A target source MUST authenticate the workload and independently authorize the subject/delegation. Its effective permission MUST be the intersection of requesting/delegated principal, source entitlement, resource/tenant boundary, data classification, purpose/consent, residency, product/process contract, and current policy. Missing, invalid, unrecognized, cross-tenant, or mismatched values MUST deny the request. Broad orchestration credentials MUST NOT substitute for a source-specific grant.

Access tokens SHOULD be short-lived and audience restricted to the protected source resource. RFC 8707 specifies the OAuth `resource` indicator and RFC 9700 recommends least privilege, audience restriction, and refusing tokens that are not intended for the resource server. [4] [5] Tokens MUST NOT be put in a URL. A target MUST NOT accept or forward a bearer token minted for another audience. Sender-constrained tokens SHOULD be used where deployment supports them.

Authentication, source authorization, policy authorization, and human approval are four distinct decisions. OAuth consent, a chat acknowledgement, an A2A `AUTH_REQUIRED` state, a user-interface checkmark, or a model statement is not automatically a human approval or policy grant. A source MAY require step-up authentication and separation-of-duties evidence; its Process Capability Contract MUST state this requirement.

7. Policy Decisions, Evidence, and Decision Specifications

A **Policy Decision** is an immutable record of an evaluated policy version, context, evaluation time, outcome, decision digest, and obligations. Its outcomes are `allow`, `allow-with-human-approval`, `deny`, or `indeterminate`. It is evidence about whether policy permits a proposed path; it is **not** a Policy Grant, Human Approval, or Authority to mutate state. `deny` and `indeterminate` MUST fail closed for action. A Human Approval or bounded Policy Grant binds the exact plan after policy evaluation.

An Evidence Bundle MUST pin each Data Product Contract, semantic references, query/compiled-plan digest, policy reference, observation time, source version/ETag or equivalent when supplied, watermark/snapshot identifier when supplied, quality/freshness state, cache/materialization declaration, and protected value reference or allowed value payload. A multi-source bundle MUST use `observational` unless a named, supported coordinated-snapshot mechanism covers all listed sources. Per-source concurrency tokens reduce stale writes; they do not imply a common read timestamp, serializable decision, or distributed atomic commit.

A Decision Spec MUST include Execution Context, accountable owner, decision question, objective, hard constraints, alternatives or eligible population, evidence references/cutoff, deterministic method type/version/parameter digest where relevant, assumptions, uncertainty, recommendation, and action eligibility. It MUST not use an LLM's unrecorded reasoning as the final consequential calculation, eligibility, allocation, or optimizer. An LLM MAY propose candidates, draft explanations, or assist a human. Deterministic rules, formulas, and optimizers SHOULD be reproducibly versioned and validated.

The decision record is not authority to mutate state. A Decision Spec may identify an action as eligible; only an approved/authorized Execution Plan and a source-side validation can permit the effect. If evidence is stale, inconsistent, semantically unresolved, insufficient for a hard constraint, or outside declared quality, action eligibility MUST be `ineligible` or `manual-review-required` unless the named domain policy states a safe alternate path.

8. Execution Plan and canonicalization

An Execution Plan MUST be an immutable ordered DAG. Its `planDigest` MUST cover a canonical `plan_core` that includes tenant, subject/workload/executor/delegation, purpose, contract/profile versions, Decision Spec reference, policy decision reference, workflow/correlation identity, execution window, and the ordered command steps. Each step MUST include capability contract version/digest, command binding, exact source audience, resolved resource target, normalized effect-bearing payload or a binding payload digest, expected source-state precondition, idempotency domain/key and invocation fingerprint, evidence references/observation commitments, required authority, risk/predicted effect, dependencies, expiry, and either a named compensation capability or a `non-compensable` declaration with accountable manual-recovery owner.

The required digest form is:

```
plan_digest = base64url(SHA-256(UTF-8(JCS(plan_core))))
```

For this v0.1 schema, `plan_core` MUST be the complete `ExecutionPlan` artifact with exactly `/header/contentDigest` and `/body/planDigest` removed; all remaining header and body fields, ordered steps and body extensions are covered. Base64url MUST be unpadded and encode the 32 digest bytes as 43 characters. Compute `planDigest` first, insert it into the body, then compute the artifact content digest using Section 4. No implementer-selected projection or additional field exclusion is allowed in this profile.

JCS means RFC 8785 JSON Canonicalization Scheme. [6] The plan core MUST include all fields that change who, what, where, when, why, or how much is affected. Transport headers, bearer tokens, detached signatures, attempt counters, trace fields, receipts, and values created after approval MUST remain outside the immutable plan artifact. Human-readable effect text MUST be included in the approval view or deterministically derived from the same plan core. It MUST NOT be a redacted presentation of a different executable payload.

Implementations MUST reject duplicate JSON member names, non-I-JSON input, non-finite numbers, and ambiguous canonicalization inputs. Exact currencies, precision quantities, timestamps, and large identifiers MUST use profile-defined canonical strings rather than IEEE-754 JSON numbers. JCS sorts object keys; array order remains semantic. JCS does not normalize arbitrary Unicode or URI values. Domain profiles MUST define identifier and value normalization before hashing and MUST NOT normalize two authorization-bound identifiers into equivalence after validation.

If a secret changes an effect, the plan MUST contain a sealed source reference/version and effect-binding commitment, not a raw secret. The approval display MUST accurately disclose the effect at the required risk level. A digest supplies integrity binding for declared bytes only; it does not prove truth, source authority, current state, semantic correctness, or resistance to prompt injection.

9. Authority, approval, expiry, and revocation

An Authority artifact is a verifiable record, not a UI flag. A policy grant MUST bind the Execution Context, exact plan reference/digest, source audience, allowed capability versions/transitions, resolved resource boundary, purpose, maximum amount/quantity/rate where applicable, risk tier, evidence requirement, issue time, expiry, revocation/status reference, and escalation policy. It is bounded, single-purpose, and revocable.

A Human Approval MUST bind the authenticated approver, exact plan reference/digest, policy decision reference, permitted step IDs, effect summary, issuance, expiry, and single-use condition. The authority service MUST validate and atomically consume it immediately before its first authorized effect. A changed target, payload, expected state, source/capability version, evidence, policy, delegation, or expiry MUST invalidate approval for the changed plan. An approval MUST NOT be reused as a generic tool permission.

Consumption MUST atomically bind the approval ID to the one durable workflow/correlation ID, exact plan digest and allowed step set. Repeated requests for that same binding MAY reconcile the existing authorized workflow; a different binding MUST fail. The binding MUST be persisted before dispatch. This atomic operation is local to the authority service: it does not atomically commit the source effect or guarantee that an effect will occur. A crash after consumption MUST recover the same binding and workflow, not consume the approval for a new plan or create a new invocation key. The binding does not extend expiry or suppress revocation checks.

Approval of a multi-step plan authorizes one durable workflow instance, not unrestricted execution of all future steps. Before each side effect, the execution service MUST derive or validate a step-scoped authorization restricted to workflow ID, step ID, source audience, capability version, resolved target, payload/precondition fingerprint, authority references, expiry, and nonce/serial. The target source MUST validate it and current source authorization immediately before effect.

Revocation MUST name an authoritative status source, distribution mode, maximum propagation interval, and audit event. Executors MUST halt queued or future steps when revocation is observed. Sources MUST check current authority/revocation status at dispatch and immediately before commit where operationally feasible. Revocation prevents later use; it cannot undo a completed source effect. Completed effects remain in the evidence record and follow a declared compensation or accountable recovery route.

10. Source state, idempotency, and outcome recovery

A side-effecting command MUST accept an expected current-state token and evaluate it at the source immediately before effect. HTTP sources SHOULD use strong ETags with `If-Match` where available. RFC 9110 specifies that `If-Match` is evaluated before a state-changing method and a false condition prevents the method from being applied. 7 A non-HTTP source MAY use an equivalent declared mechanism. It MUST publish the comparison semantics. A failed check MUST return `STALE_PRECONDITION` or `PRECONDITION_FAILED`, identify the safe current status/version where permitted, and make no new effect.

For each hard business invariant involving another source, the Process Capability Contract MUST declare how that invariant is protected at execution: a source-owned authoritative guard, a transactional reservation/coordination mechanism with declared guarantees, or an explicitly permitted domain risk boundary that does not misrepresent a hard invariant as guaranteed. A local ETag MUST NOT be presented as protection for independently changing external facts that it does not cover. If the required guarantee has no enforcing mechanism, the executor MUST reject the action as unsupported or require a declared safe

alternative. Re-reading multiple sources immediately before dispatch does not by itself eliminate the race or create atomicity.

Each command MUST declare idempotency domain, minimum key length/entropy, retention interval, payload/invocation fingerprint algorithm, duplicate result semantics, and operation-status lookup. Within the declared domain, the same key plus the same fingerprint MUST return the original operation/receipt reference without a second effect. The same key plus a different fingerprint MUST return `IDEMPOTENCY_KEY_REUSE_CONFLICT` without effect. Implementations SHOULD use `SHA-256(JCS(invocation_core))` for a documented invocation core containing tenant, capability/version, resolved target, normalized effect-bearing payload, and source-enforced precondition, while excluding credentials, timestamps, trace fields, and attempt number.

After a timeout, disconnect, worker crash, or ambiguous server response, the executor MUST record an unknown-pending-reconciliation state and query source operation status and/or idempotency status before any resend. It MAY retransmit only the same authorized invocation and key if the source contract declares the retransmission safe. It MUST NOT generate a fresh idempotency key, assume no effect, or claim completion from transport success. HTTP's method-level retry rules do not eliminate the need for this source-specific behavior. [7]

Operation Receipts MUST be source-issued and immutable. They MUST identify invocation, operation ID, receipt sequence, predecessor receipt where relevant, observation time, outcome, effect/status, resource version or postcondition evidence where relevant, safe error, and reconciliation route. A receipt changes through a new linked receipt; it is not overwritten.

11. Lifecycle, cancellation, compensation, and errors

The normal lifecycle is:

`SUBMITTED → (NEGOTIATING | GATHERING_EVIDENCE) → (AWAITING_INPUT | AWAITING_AUTHORITY | PLANNED) → AUTHORIZED → EXECUTING → VERIFYING → COMPLETED`

Terminal outcomes are `REJECTED`, `EXPIRED`, `FAILED`, `CANCELED`, `PARTIALLY_COMPLETED`, `UNKNOWN_OUTCOME`, and `MANUAL_RESOLUTION_REQUIRED`. A participant MAY expose finer-grained internal states only when it preserves these meanings and links all associated evidence.

`COMPLETED` requires required source receipts and verified postconditions. `FAILED` requires evidence of no effect for all relevant steps, otherwise the workflow MUST use `PARTIALLY_COMPLETED`, `UNKNOWN_OUTCOME`, or `MANUAL_RESOLUTION_REQUIRED`. Cancellation is a request, not proof of no effect. `CANCELED` requires evidence that no effect occurred. A cancellation that arrives after one or more effects MUST be represented by a receipt outcome such as `cancelled-after-effect` and reconciled to a truthful terminal workflow state.

A multi-source plan MUST declare one of three paths before its first potentially partial effect: named source-owned compensations; explicit non-compensability and a named accountable manual-recovery owner; or a source-local atomic group. A compensation is a new business command with authority, precondition, idempotency, receipt, and outcome. It is not a time machine. A saga provides eventual consistency and can expose intermediate states; compensation can fail or be inappropriate for payments, physical shipment, notices, or disclosures. [8] A failed compensation MUST end in `MANUAL_RESOLUTION_REQUIRED` unless evidence proves a different safe result.

Errors MUST carry a stable code, task/correlation identifier, retry classification, safe detail, and retry-after guidance where relevant. They MUST NOT expose secrets, raw unapproved data, hidden policy reasoning, or unnecessary sensitive evidence. The v0.1 vocabulary is: `INVALID_ARTIFACT`, `UNSUPPORTED_PROFILE`, `UNSUPPORTED_VERSION`, `UNKNOWN_CRITICAL_EXTENSION`, `UNTRUSTED_PARTICIPANT`, `AUTH_REQUIRED`, `AUTH_DENIED`, `PURPOSE_DENIED`, `NEGOTIATION_REQUIRED`, `APPROVAL_REQUIRED`, `AUTHORITY_EXPIRED`, `AUTHORITY_CONSUMED`, `EVIDENCE_STALE`, `PRECONDITION_FAILED`, `STALE_PRECONDITION`, `CONFLICT`, `IDEMPOTENCY_KEY_REUSE_CONFLICT`, `QUOTA_EXCEEDED`, `POLICY_EVALUATION_FAILED`, `OPERATION_UNKNOWN`, `CANCEL_NOT_POSSIBLE`, and `INTERNAL`.

12. Privacy, residency, logging, and untrusted content

Publishers and executors MUST enforce contract-declared classification, purpose, retention, residency, and entitlement at the source boundary. They MUST minimize logs. Operational logs SHOULD store correlation/task IDs, artifact IDs/digests, safe status, timing, policy/result references, and receipt references. They MUST NOT normally store raw sensitive payloads, bearer tokens, secrets, hidden model prompts, or full protected evidence. Retention, encryption, redaction, audit-access roles, cache behavior, and cross-region movement MUST follow the governing product/process contract and deployment requirements.

User input, retrieved data, documents, tool descriptions, prompts, source text, application-builder output, and inter-agent messages MUST be treated as untrusted instructions. A signature or publisher identity may attest origin and integrity of bytes; it does not make content true, current, authorized, or safe to follow. OWASP guidance specifically says execution-side controls must hold authority outside agent context and calls out retrieved/API content as injection-capable. 9 The trusted control plane MUST ensure that untrusted content cannot modify policy, plan core, source authority, state precondition, or source invariants.

13. Application Specifications and builder prompts

An Application Spec is a canonical, typed presentation contract. It MUST pin its Decision Spec, exact Execution Plan, allowed evidence views, reviewed component catalog version, component set and typed bindings, user-input schema, classification/accessibility requirements, and action-event-to-plan-step mapping. It MUST NOT contain arbitrary executable code, scripts, unbounded SQL, hidden source credentials, arbitrary URLs, or implicit source invocation.

A Builder Prompt is a derived, non-authoritative artifact. It MUST identify its Application Spec reference/digest, target builder/runtime, compiler/template version, permitted component/action catalog, requested output, and no-secret declaration. A v0/Vercel-like builder MAY consume it. Its generated code MUST still pass ordinary review, source integration checks, dependency/security assessment, tests, ownership assignment, and release control. It MUST NOT change policy, entitlement, plan digest, authority, receipt semantics, or source validation.

This separation permits portable application generation without making prompt text a control plane. It reflects the limitation documented by structured-output tooling: a schema can validate shape but model output can still be incorrect or incomplete. 10

14. Bindings and compatibility

APS-DDA runs over HTTPS. A direct JSON binding MAY carry a profile URI and required in-body profile version. The following routes are illustrative proposed adapter conventions only, **not** registered

or existing APS-DDA, A2A, or MCP methods: `GET /.well-known/aps-dda/manifest`, `POST /aps/v0.1/tasks`, `POST /aps/v0.1/commands:invoke`, and `GET /aps/v0.1/operations/{id}`. An actual endpoint MUST be declared in the trusted Capability Manifest.

For MCP, an adapter MAY expose typed tools such as `aps.resolveArtifact`, `aps.submitTask`, and `aps.invokeCommand`, with inputs and outputs conforming to this profile's schemas. This document does not claim those are existing MCP methods. MCP **2026-07-28** defines tools with JSON Schema input and optional output schema, and its Tasks capability is a negotiated `io.modelcontextprotocol/tasks` extension. 11 12 An APS-DDA MCP binding MUST pin `2026-07-28` and each required extension, negotiate them explicitly, and return an Operation Receipt plus polling route if Tasks is unavailable. MCP tool call/response mechanics do not replace APS-DDA authority or outcome rules.

For A2A, an adapter MAY carry APS-DDA records as typed data/artifacts and map progress to A2A Tasks. A2A **v1.0** is documented in the official 12 March 2026 release announcement as its first stable production-ready release. 13 A binding MUST pin the selected A2A version and binding rather than rely on a mutable `latest` URL. Generic A2A task states and `AUTH_REQUIRED` are useful transport events but do not change APS-DDA requirements for plan-bound authority, source receipts, or unknown/partial outcomes.

Manifests MUST advertise supported profile major/minor and binding versions. Unsupported major versions or unknown critical extensions MUST be rejected. Extensions MUST be URI-named, versioned, and declared critical or non-critical. Core objects SHOULD be closed outside their extensions object. A participant MAY ignore an unknown non-critical extension only if doing so does not change a required business constraint or effect.

15. Conformance tests and non-guarantees

Conformance MUST be tested in three distinct groups.

Syntactic tests validate JSON Schema shape, discriminators, required artifact references, version negotiation, closed fields, and canonicalization vectors. The supplied `test_schema.py` is only a lightweight structural example. A real implementation MUST add its chosen signature/digest and media-type tests.

Behavioral tests exercise scope negotiation, refusal of unsupported profile/critical extension, approval expiry and single-use race, policy failure and revocation, stale-state rejection, duplicate key same/different payload behavior, timeout after possible success, crash/restart, operation reconciliation, loss/duplication/reordering of notifications, cancellation races, partial failure, compensation failure, log redaction, and durable terminal-state evidence.

Semantic tests validate the published domain binding, identity/grain/time/currency/calendar compatibility, source-enforced invariants, data-product meaning, and receipt postconditions. They must include adversarial data/tool/document text intended to broaden scope or bypass authority. A compliant executor MUST treat that content as evidence or quarantine it; it must not change deterministic controls.

Passing any of these groups establishes only compliance with stated fixtures and profile behavior. It does not certify data truth, completeness, security against every attack, lawful processing, business success, or correctness of arbitrary decisions. No participant may claim a universal truth guarantee from APS-DDA conformance.

16. Implementation boundary

APS-DDA v0.1 is designed to profile existing HTTPS interactions before proposing a new transport. MCP and A2A are useful substrates; source agents are useful participants; deterministic adapters may be preferable to autonomous agents for many capabilities. The profile asks for the additional commitments that make a bounded operational decision loop inspectable and recoverable.

The profile does not remove the need for architecture judgment. A workflow MUST remain on an existing governed analytical, integration, or workflow product when its requirements include unconstrained scans, recurring high-volume joins that exceed source budgets, unresolved identity/history requirements, a global atomic snapshot, cross-source atomic commit, training/long-horizon BI, or irreversible action without accountable recovery. Those boundaries protect the replacement thesis from promising what a behavioral profile cannot deliver.

References

Bibliographic entries

1. **RFC 2119: Key words for use in RFCs to Indicate Requirement Levels** — https://www.rfc-editor.org/rfc/rfc2119
2. **RFC 8174: Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words** — https://www.rfc-editor.org/info/rfc8174/
3. **JSON Schema Draft 2020-12 Release Notes** — https://json-schema.org/draft/2020-12/release-notes
4. **RFC 8707: Resource Indicators for OAuth 2.0** — https://www.rfc-editor.org/rfc/rfc8707.html
5. **RFC 9700: Best Current Practice for OAuth 2.0 Security** — https://www.rfc-editor.org/rfc/rfc9700.html
6. **RFC 8785: JSON Canonicalization Scheme** — https://www.rfc-editor.org/rfc/rfc8785.html
7. **RFC 9110: HTTP Semantics** — https://www.rfc-editor.org/rfc/rfc9110.html
8. **Temporal Saga Pattern** — https://docs.temporal.io/design-patterns/saga-pattern
9. **OWASP AI Agent Security Cheat Sheet** — https://cheatsheetseries.owasp.org/cheatsheets/AI_Agent_Security_Cheat_Sheet.html
10. **AI SDK: Generating Structured Data** — https://ai-sdk.dev/docs/ai-sdk-core/generating-structured-data
11. **MCP Server Tools, revision 2026-07-28** — https://modelcontextprotocol.io/specification/2026-07-28/server/tools
12. **MCP Tasks Extension Overview** — https://modelcontextprotocol.io/extensions/tasks/overview
13. **A2A Protocol Ships v1.0: Production-Ready Standard for Agent-to-Agent Communication** — https://a2a-protocol.org/latest/blog/2026/03/12/a2a-protocol-ships-v10-production-ready-standard-for-agent-to-agent-communication/